

OOPs Mid Term Examination

26/09/2024

Ques.1: Java vs C++

- Java and C++ are both OOPs languages.
- Java doesn't support pointers.
- Java doesn't support global variables as nothing is declared globally, though it can be achieved through Inheritance.
- Java doesn't support Multiple Inheritance, but it can be achieved through interface.

Ques.2: Java vs C

- C is structured language while Java is OOPs language.
- C supports pointers, java doesn't.
- Java doesn't have pre-processors like C.
- Java doesn't include structures.
- Java doesn't include keywords like goto, size, typedef.
- Java adds labelled break and continue.

Ques.3: What is JVM?

- ➔ CPUs are capable of only executing machine language.
- ➔ Different types of CPUs have different sets of machine instructions.
- ➔ Java compiles programs into machine instructions to enable it to be understood and executed by an idealized CPU. Such CPUs are known as Java Virtual Machines or JVM.

Ques.4: What is Byte code? Explain JIT.

- ➔ Java compiles programs into machine code that can be understood by JVM, such a code is called Byte code. Although the details of JVM may differ on platform-to-platform basis but they all interpret Java byte code into native machine code. JIT or Just In Time is a java compiler which compiles Java Byte Code into native machine code in real time on a piece by piece demand basis.

Ques.5: What is JRE and JDK?

- ➔ The Java Runtime Environment (JRE) is a software layer that allows Java programs to run on any operating system without modification. It's a key component of the Java programming language and is made up of several components, including: JVM, Java class libraries, Java class loaders, JDBC (Java Database Connectivity), etc.
- ➔ JDK or Java Development Kit is a cross-platform software development environment which is used to create java applications. It includes the JRE (Java Run-time Environment), JVM (Java Virtual Machine), the java language and the compilers along with other tools and libraries needed to successfully write, compile, debug and execute a java program.

Ques.6: Difference between Procedural and Object-Oriented Programming.

→ Procedural:

- Larger programs are divided into smaller programs known as functions or modules.
- Data is not hidden and it can be accessed by external functions.
- Follows top-down approach in program design.
- Data may communicate with each other through functions.
- Emphasis is on procedure rather than data.

→ Object-oriented:

- Programs are divided into objects.
- Data is hidden and it can't be accessed by external functions.
- Follows bottom-up approach in program design.
- Objects may communicate with each other through functions.
- Emphasis is on data rather than procedure.

Ques.7: What are some major concepts of Object-oriented programming?

- **Objects:** Objects are basic building blocks for designing programs. It is a collection of data members and associated member functions. Each object is identified by a unique name. Objects must be part of a class.
- **Classes:** Class is a user defined data type which holds its own data members and member functions which can be accessed by creating an instance of that class. Classes are used to create new objects and manage them, organize information and support inheritance.
- **Inheritance:** Inheritance in OOPs allows one class to inherit the features (fields and methods) of another class. This eliminates the need to write the same set of codes repeatedly for different classes. We implement inheritance with the help of "extends" keyword in java. Class inheriting the features is known as derived class or child class while the class whose features are being inherited is known as base class or parent class.
- **Data Encapsulation:** Encapsulation can be defined as binding of the data and the related member functions that manipulate them. Data encapsulation leads to data abstraction and data hiding. Advantages of data encapsulation includes security, modularity, reusability, simplifies the code, easier debugging and testing.
- **Data Abstraction:** Data abstraction is a very crucial feature of OOPs. It allows programs to display only the essential information about the data and hiding the background details such as implementation. Advantages of data abstractions are similar to that of data encapsulation, it allows programmers to make changes in the implementation of a class without affecting the user.
- **Polymorphism:** In OOPs, an operation can have different results in different instances. The behaviour of an operator depends upon the type of data type used in the operation. Java mainly supports two types of polymorphism, namely Compile-time Polymorphism and Run-time Polymorphism.
 - **Compile-time:** Also known as Static Polymorphism, it is achieved by function/method overloading. Method overloading occurs when there are multiple methods with the same name but different parameters.
 - **Run-time:** Also known as Dynamic Method Dispatch, it is achieved when a function call to an overridden method is referenced at runtime. Method overriding occurs when a derived class has a definition for one of the member functions of the base class.

- ➔ **Dynamic Binding:** In dynamic binding, the code to be executed in response to the function call is decided at runtime. Although Dynamic binding is slower than static binding it avoids many of the drawbacks of static binding such as reduced flexibility, need to recompile, errors and increased compilation time.
- ➔ **Message Passing:** Objects communicate with each other by sending and receiving information. A message for an object is a request for the execution of a procedure and therefore will invoke a function in the receiving object that generates the desired result. Message passing involves specifying the name of the objects, name of the function and the information to be sent.

Ques.8: What are some of the advantages and disadvantages of OOPs over Procedural programming?

- ➔ **Advantages:** Code reusability, easier to understand, easier to modify, faster development, modularity, inheritance, security.
- ➔ **Disadvantages:** Complexity (with the inclusion of multiple level of inheritance, polymorphism & dynamic binding), Performance (less efficient in terms of CPU & memory usage), Libraries (OOPs requires many packages & libraries to be installed for the code to work which can be confusing for some developers).

Ques.9: What are primitive and non-primitive data types?

- ➔ **Primitive:** A primitive data type specifies the size and type of variable values, it has no additional methods. They are pre-defined. Ex: int, char, double, float, Boolean, etc.
 - Int: 4 bytes, $(-2^{31} \text{ to } 2^{31} - 1)$
 - Char: 2 bytes
 - Float: 4 bytes
 - Double: 8 bytes
 - Boolean: 1 byte
- ➔ **Non-primitive:** Also known as reference types because they refer to objects. They can be used to call methods and they can be null unlike primitive data types. It is user defined. Ex: Class, String, Arrays, Interface, etc.
 - **Class:** Template or blueprint of an object where object can be anything from real-world.


```
public class Cat
{
    color;
    breed;

    getbreed();
    meow();
    eat();
}
```
 - **String:** It represents a collection of characters, it is an object of type java.lang.String The class contains methods that can perform certain operations on string.


```
String s= "hello"
System.out.println("The length of s is: " + s.length()); //Output: The length of s is: 5
```
 - **Arrays:** Arrays are used to store multiple homogeneous values in a single variable. In java, array index starts from 0 (zero).

```
String fruit[] = ["apple", "banana", "mango"];
System.out.println("The third fruit is: " + fruit[2]); //Output: The third fruit is: mango
```

- **Interface:** Interface is a non-primitive data type which is used to specify the contracts or capabilities that implementing classes must implement. An interface has abstract methods with only signatures which are then implemented by some classes. They are used to achieve abstraction, polymorphism and multiple inheritance.

```
interface Security
{
    boolean accessCheck(String employeeToken);
}
```

Ques.10: What are the different types of variables in Java?

- ➔ **Local Variable:** Declared inside methods, constructors or blocks. Access modifiers such as public, private, protected cannot be used for local variables. Only visible within the declared method/constructors/blocks. Implemented at stack level internally. Doesn't have a default value so it should be declared and initial value should be assigned before first use.
- ➔ **Instance Variable:** Declared in a class but outside a method, constructor or block. Access modifiers can be used. When space is allocated for the object in heap, a slot for each instance variable value is created with it. It holds value that must be referenced by more than one method/constructor/block or essential parts of an object's state that must be present throughout the class.

```
➔ public class Employee
➔ {
➔     public String name; //this instance is visible for any child class
➔     private double salary; //this instance is only visible for Employee class only
➔     public Employee(String empName)
➔     {
➔         name = empName;
➔     }
➔     public void setsalary(double empSal)
➔     {
➔         salary = empSal;
➔     }
➔     public void printEmp()
➔     {
➔         System.out.println("name: " + name);
➔         System.out.println("salary: " + salary);
➔     }
➔     public static void main(String args[])
➔     {
➔         Employee e = new Employee("Rajesh");
➔         e.setsalary(1000);
➔         e.printEmp();
➔     }
➔ }
➔ /*Output: name: Rajesh
➔           salary: 1000.0*/
```

➔ **Class/Static Variable:** Declared with “static” keyword in a class but outside method, constructor or block. Static variables are rarely used other than being declared as constants. There would only be one copy of each class variable per class. Stored in static memory. Access modifiers can be used.

```
➔ public class Employee
➔ {
➔     private static double salary=500; //salary variable is a private static variable
➔     public static final String Department= "Development";
➔     public static void main(String args[])
➔     {
➔         salary+= 1000;
➔         System.out.println(Department);
➔         System.out.println(salary);
➔     }
➔ }
➔ /*Output:
➔ Development
➔ 1500.0
➔ */
```

Ques.11: What are Java Tokens?

➔ In Java, **Tokens** are the smallest elements of a program that is meaningful to the compiler. They are also known as the fundamental building blocks of the program. Tokens can be classified as follows:

- **Keywords:** Keywords are pre-defined or reserved words in a programming language. They are meant to perform a specific function. Keywords cannot be used to name a variable.
- **Identifiers:** Identifiers are used to name variables in a program. An identifier must only contain alphanumerical values and special characters ‘_’ (underscore) & ‘\$’ (dollar sign). It is user-defined and the first character of an identifier cannot start with a digit.
- **Constants:** Constants are such values that do not change throughout the program execution. In java, constants are declared using “final” keyword (public static final var_name= var;).
- **Special Symbols:**
 - **Brackets []** Brackets are used to reference array elements. (For example, ar[3]= [“apple”, “banana”, “mango”];). Multidimensional arrays are declared using multiple brackets.
 - **Parentheses ()** These special symbols are used to indicate function calls and function parameters.
 - **Braces { }** Opening and closing curly braces indicates start and end of a block of code, containing more than one executable statement.
 - **Comma (,)** Used to separate more than one statements or parameters in function calls.
 - **Semi colon (;)** Semi colons are used to terminate a statement.
- **Operators:**
 - **Arithmetic:** (+, -, /, *, %)
 - **Assignment Operator (=):** Used to assign values to a variable.
 - **Logical:** AND (&&), OR (||), NOT (!).
 - **Relational:** Equal to (==), Not Equal to (!=), Greater than (>), Less than (<), Greater than or Equal to (>=), Less than or Equal to (<=)
 - **Unary:** Unary minus(-), Not(!), Post and Pre Increment (i++, ++i), Post and Pre Decrement (i--, --i), Bitwise Complement(~).
 - **Ternary:** condition check ? result 1(if True) : result 2 (if False).
 - **Bitwise:** Bitwise OR (|), Bitwise AND (&), Bitwise XOR (^), Bitwise complement (~).

- **Shift:** Shifts the bits of operand left or right by a given number (<<, >>, <<<, >>>).
- **Comments:** Comments in java are ignored by compiler during execution. They are non-executable piece of statement in a program used to describe a method or a block of code.
- **Separators:** Separators are used to separate different parts of code. It tells the compiler about the completion of a statement in a program.

Ques.12: What is Method Overloading and Method Overriding?

- ➔ **Method Overloading:** Method Overloading can occur in two ways, first when methods with same name have different number of parameters. For ex: add(int num1, int num2) & add(int num1, int num2, int num3). Secondly it can occur when method have same name but different data type for their parameters. For ex: add(int num1, int num2) & add(double num1, double num2).
- ➔ **Method Overriding:** Method overriding occurs when a child class has a method with same name as that of a parent class method and same parameter as well. For ex:

```
class Vehicle
{
    void run()
    {
        System.out.println("Vehicle is running");
    }
}
class Bike extends Vehicle
{
    void run()
    {
        System.out.println("Bike is running");
    }
}
```

Ques.13: What are access modifiers and explain their types?

- ➔ Access modifiers in Java are used to give different level of access to certain data member or methods of that class to the code outside of that class. There are 4 types of Access Modifiers in Java:
 - **Private:** The access level of a private access modifier is only limited to that class and nothing outside of that class can access it.
 - **Default:** The access level of a default modifier is only within that package and it cannot be accessed from outside that package.
 - **Protected:** The access level of a protected modifier is within and outside the package made possible through sub classes.
 - **Public:** The access level of a public modifier is everywhere. It can be access within the class, outside the class, within the package and outside the package.

Ques.14: